

PARIKSHAK — project documentation

On-board procedure witness for crewed space missions

Smart India Hackathon 2026 · Problem Statement **26174** · Space Technology · Software Problem statement: *AI-based Human Activity Recognition for validating scientific experiment sequences on-board (BAS / lunar missions)* — Indian Space Research Organisation (ISRO), Department of Space.

Status of this document: 22 September 2026. Every number in it is read from a file in `runs/`, named where it is quoted, and can be regenerated with the commands in section 14.

Contents

1. [Summary](#)
 2. [The problem](#)
 3. [What PARIKSHAK does](#)
 4. [Architecture](#)
 5. [Core concepts](#)
 6. [The two experiments](#)
 7. [Software components](#)
 8. [Running it](#)
 9. [Adding a new experiment](#)
 10. [How it is evaluated](#)
 11. [Results](#)
 12. [Performance and endurance](#)
 13. [Testing and quality gates](#)
 14. [Deliverables and how to regenerate them](#)
 15. [Honest limits](#)
 16. [Road to a flight system](#)
 17. [Licences](#)
 18. [Glossary](#)
-

1. Summary

An astronaut runs a science procedure on the space station. PARIKSHAK watches the experiment rack through a camera, checks every step against the procedure file, speaks the next step, and raises an alert only when the evidence is strong — always naming the step and the reason. When the camera cannot see, it says "cannot verify" instead of guessing. Every run writes a small, tamper-evident flight record that is sent to the ground before any video.

What exists today is a working software prototype:

- a procedure engine that verifies steps, detects skipped steps, wrong objects, timing violations and safety hazards, and explains every decision;

- two experiments written as files — CSP-1 (14 steps) and CRX-2 (8 steps) — the second added with no retraining and no code change;
- a browser app in which you choose an experiment and are guided through it while the engine judges each step live, plus replays of 14 recorded runs;
- a desktop operator window, a live webcam mode using printed markers, voice prompts, video clips around alerts, and a hash-chained log with a verifier;
- an evaluation on 200 degraded runs, a laptop CPU benchmark and a 45-minute endurance test, all reproducible.

Measured on the degraded evaluation corpus, **every target is met on both experiments**:

Measure	Target	CSP-1 (130 runs)	CRX-2 (70 runs)
Step accuracy	$\geq 95 \%$	98.7 %	95.6 %
Deviations caught	$\geq 90 \%$	97.8 %	97.5 %
False alarms per 45 min	≤ 1	0.85	0.00
False alarms on permitted reorders	0	0	0
Alert delay, median / 95th percentile	$\leq 2 \text{ s}$	1.6 / 1.6 s	1.6 / 1.8 s

Sources: `runs/eval.json`, `runs/eval_crx2.json`.

2. The problem

Who it is for. The crew member at the rack — hands busy, eyes on the work, no keyboard — and the ground principal investigator who reads the record weeks later and must decide whether the run is valid.

Why the ground cannot do it. A round trip to the Moon takes about 2.6 s and to Mars 6 to 44 minutes. Ground control cannot watch a step as it happens, and on the Bharatiya Antariksh Station (BAS) or a lunar mission the link will not carry continuous video of every experiment. The witness has to be on board.

Why a checklist is not enough. Today the crew ticks each step on a tablet. That costs exactly the attention the procedure needs, and self-reporting cannot catch a mistake the crew did not notice: a latch left open, the wrong vial picked from two that look alike, a hazard that skips no step at all.

The hidden requirement. An assistant that raises false alarms is switched off. The design therefore treats a false accusation as the most expensive error: missing evidence must never become an alarm, and every alarm must be defensible afterwards.

3. What PARIKSHAK does

It...	How
Guides the crew	Speaks each step's prompt as it becomes current; shows a live checklist
Verifies each step independently	Evaluates the step's conditions against what the camera sees, over time
Catches skipped steps	Notices a later step completing while this one never did, with evidence against it
Catches the wrong object	Watches for the confusable object the procedure itself declares (vial A vs vial B)
Catches hazards	Safety rules that must hold throughout a step, such as "latch closed while the unit runs"
Catches timing problems	A step done faster than physically plausible, or far slower than its limit
Handles hardware faults	Follows the procedure's fault branch instead of blaming the crew
Admits blindness	Reports "cannot verify" when the view is blocked or the rack is lost — never "skipped"
Explains itself	Every alert carries the condition that failed and when
Keeps a flight record	A SHA-256 hash-chained log of every decision; editing one byte is detected
Records evidence	A ± 10 s video clip around each alert (live runs)
Runs offline	No module in the on-board software can open a network connection

4. Architecture

Camera → Perception → Belief frame → Procedure engine → Crew and ground

(rack)	rack pose from AprilTags; objects, states, hands	what is seen, with confidence; hidden stays "unknown"	predicates, evidence over time, duration priors, safety invariants	voice prompt, alert with reason, checklist, hash-chained log
--------	--	---	--	--

The pipeline has one hard seam: the **belief frame**. Everything to its left is perception and may be replaced — printed markers today, trained detectors on a flight build. Everything to its right is the reasoning engine and never sees a pixel. A recorded run is simply a file of belief frames, so the engine can be replayed deterministically from any recording, and the evaluation, the demo and the tests all drive the same engine the live system uses.

Per frame, the data flow is:

1. The camera image goes to perception, which locks the rack frame from four AprilTags (camera pose by PnP) and reports each declared object's position in rack coordinates, its state (latch open/closed, unit idle/running) and any hands.
2. Perception emits a `BeliefFrame`: per object, visible / occluded / position / state / confidence; plus hands, motion class and crew confirmations.
3. The step tracker evaluates every step's verification tree, accumulates evidence for and against each step, and resolves steps as complete, skipped or unverified.
4. The deviation detector turns those transitions into typed deviations; the alert policy decides, with persistence and severity, what is said to the crew.
5. The run logger appends each decision to the hash-chained record; the display state is rebuilt from the engine for the screen.

5. Core concepts

5.1 The procedure is a file

An experiment is written in the procedure language (PDL, YAML validated by a JSON Schema in `schema/`). It declares the zones of the rack, the entities and their states, each step with its prompt, objects, verification condition, duration prior and optional safety invariants, unordered groups of steps, branches, and the alert policy. The perception layer learns hands and objects; it never learns an experiment. A new experiment is therefore a new file.

5.2 Three-valued truth — unknown is not wrong

Every condition evaluates to an interval `[lo, hi]` rather than a yes/no: `TRUE` is `[1, 1]`, `FALSE` is `[0, 0]`, and `UNKNOWN` is `[0, 1]`. An occluded object is unknown, not absent; a condition that needs a model the setup lacks (hand tracking, a motion classifier) is unknown, not false; geometry without a rack lock is unknown. Only positive evidence against a step can make it "skipped". Losing information may move a verdict toward "cannot verify", never toward an accusation.

5.3 Predicates

The closed vocabulary of conditions includes `visible`, `absent`, `count_of`, `inside`, `near`, `aligned`, `contacting`, `stable`, `released`, `grasped`, `hand_in_zone`, `state_is`, `motion_is`, `motion_count`, `crew_confirmed` and `assert_true`, combined with `all`, `any`, `not`, `hold_for` (held across a window, tolerating a share of disagreeing frames) and `occurred` (happened at some moment during the step). The authoritative definitions, with the capability each needs, are in `schema/PREDICATES.md`; a test keeps that document in step with the code.

5.4 Evidence over time

A step is not judged on one frame. Each step keeps two one-sided accumulators (CUSUM-style): support that it is done and evidence that it is not, updated at the procedure's evidence rate (2 independent observations per second). A step completes when support passes the completion bar (0.85); it can be called skipped only when evidence against it passes the deviation bar (0.85) **and** it was seen undone recently — within its own minimum duration. Duration priors (minimum, nominal, maximum per step) make "skipped" a judgement about time, not a timeout.

5.5 Deviations and alerts

Kind	Meaning
SKIP	A later step completed while this one had evidence against it
OUT_OF_ORDER	A step reported skipped was later completed
WRONG_OBJECT	The declared confusable object was held in this step
HAZARD	A safety invariant was violated during a step
DURATION	Completed below its minimum or above its maximum duration
REPEAT	A completed step was performed again

The alert policy maps each to a severity tier — info (screen), advisory (screen + chime), caution (screen + chime + voice), critical — and requires the deviation to persist for 1.5 s before speaking. Every alert carries the spoken text and the machine reason: for example *"Step eight, close the glovebox latch, was not completed"* with *"predicate state_is(latch, closed) never satisfied between 43.0 s and 46.2 s"*.

5.6 Orientation-agnostic geometry

AprilTag fiducials are bonded to the rack face, and every position is expressed in the rack's own frame. Up is the rack, not gravity: rotating the rack, the camera or the crew member changes none of the numbers the engine reasons with.

5.7 The flight record

Each run writes a JSON-lines log in which every record carries the SHA-256 of the previous one (`tools/verify_log.py` recomputes the chain and names the first altered record), plus a human-readable mirror. The record names the procedure's hash and the operating point. A 140.8 s run's log is 8,275 bytes against about 70.4 MB of video — the log is downlinked first (`runs/download_sample/`).

5.8 Offline by construction

`tests/test_offline.py` reads the source of every module under `parikshak/` and fails the build if any imports a module capable of opening a network connection. The browser demo is a ground-side presentation tool and lives outside the on-board package (`demo/`) for exactly that reason.

6. The two experiments

CSP-1 — Colloidal Suspension Sample Processing (14 steps, `procedures/csp1_colloid_sample_processing.yaml`): secure in the foot restraint; retrieve cartridge SC-A and vial A (an unordered pair); verify the cartridge label; open the glovebox latch; insert the cartridge into holder H1; attach vial A and rotate to seal; close the latch (critical); press START; monitor the 60-second cycle (with a safety invariant: the latch must stay closed while the unit runs, and a fault branch on a red indicator); open the latch; remove the cartridge; seal it in sample bag SB-01 (critical); stow the bag in cold locker L1 (critical).

CRX-2 — Colloid resuspension (8 steps, `procedures/crx2_colloid_resuspension.yaml`): secure in the restraint; retrieve vial B from cold locker L1; confirm the processing unit is idle; resuspend vial B by agitation (ten counted motions, with a geometric safety rule — not within 12 cm of the powered unit); place it in tray T1 to settle; confirm the suspension is uniform; return vial B to the cold locker (critical); confirm the run is complete.

CRX-2 uses the same rack, the same object classes and the same software; it was added as a file and verified against the perception build with `tools/onboard.py` — zero retraining.

7. Software components

Path	What it holds
<code>parikshak/belief/</code>	<code>BeliefFrame</code> (the contract between perception and reasoning) and trace I/O, including a streaming trace writer
<code>parikshak/pdl/</code>	Procedure loader, validator and command-line checker
<code>parikshak/engine/</code>	Three-valued truth, predicates, evidence accumulation, step tracker (<code>hsmm.py</code>), deviation detector, alert policy, run logger, <code>ProcedureEngine</code>
<code>parikshak/perception/</code>	Rack frame from AprilTags (PnP), marker-based objects and states, detector back-ends, the perception pipeline
<code>parikshak/io/</code>	Cameras (OpenCV, synthetic), GStreamer capture graph, speech queue and voices, voice-command grammar, alert clips (ffmpeg path and in-memory OpenCV path), downlink packaging

Path	What it holds
<code>parikshak/gui/</code>	Desktop operator window, display state, camera overlay
<code>parikshak/eval/</code>	World-model simulator, the two experiments' scripts, the degraded corpus, perturbation profiles, the evaluation harness, renderers
<code>parikshak/live.py</code> , <code>run.py</code> , <code>replay.py</code>	Live session, the <code>parikshak run</code> command, the replay command
<code>demo/</code>	Browser app: guided runs (<code>guided.py</code>), recorded scenarios (<code>scenarios.py</code>), web server (<code>server.py</code>), the page
<code>procedures/</code> , <code>racks/</code> , <code>schema/</code> , <code>builds/</code>	Experiments, rack and prop marker layouts, the procedure schema and predicate contract, the perception build
<code>traces/</code>	Golden runs (9 CSP-1, 7 CRX-2) and the degraded evaluation corpus (130 + 70)
<code>tools/</code>	Checks, evaluation and ROC sweep, benchmark, endurance test, power report, reel, tag sheets, downlink sample, demo page, the deck
<code>deploy/jetson/</code>	Edge bring-up package — written, not yet run on hardware
<code>tests/</code>	602 automated tests

8. Running it

Install (Python 3.12 or newer):

```
pip install -e ".[web,gui,dev]" opencv-contrib-python pyttvx3
```

The engine alone needs only NumPy, PyYAML and jsonschema.

The browser app — the fastest way to see everything:

```
python -m demo
```

- *Perform an experiment:* choose CSP-1 or CRX-2. At each step choose what the crew does — do the step, skip it, take the wrong item, open the glovebox while the unit runs, work with

the camera blocked, or wait. The simulated rack responds and the real engine judges it; at the end you get the summary and the verified record.

- *Watch a recorded run*: 14 runs replayed with playback, timeline and record.
- *How it works* and *Results & limits*.

`python tools/make_demo_page.py` writes `runs/demo/parikshak_demo.html`, a single file with every recorded run embedded that opens offline by double-click.

Other modes:

What	Command
Desktop window replaying a run	<code>python -m parikshak.replay traces/golden/skip_S08_latch.jsonl --window</code>
Terminal replay, spoken	<code>python -m parikshak.replay traces/golden/skip_S08_latch.jsonl --speak</code>
Live run on a rendered camera	<code>python -m parikshak.run --camera synthetic --procedure procedures/crx2_colloid_resuspension.yaml --props racks/props_crx2.json</code>
Live run on a webcam	print <code>runs/tags/tags.pdf</code> at 100 %, then the same with <code>--camera 0</code>
Verify a run log	<code>python tools/verify_log.py runs/live/<run>.log.jsonl</code>

A live run writes its log, its trace and a ± 10 s clip around each alert into `runs/live/`.

9. Adding a new experiment

1. Copy `procedures/_template.yaml` and describe the zones, entities (with `confusable_with` where two objects can be mistaken), steps, verification conditions, duration priors, invariants and alert policy.
2. Validate: `python tools/validate_procedure.py procedures/<new>.yaml`. The validator checks the schema, predicate arguments, the step graph, duration sums and capability requirements, and prints the duration priors.
3. Check it needs no retraining: `python tools/onboard.py procedures/<new>.yaml` confirms every object class it uses is in the perception build.
4. Optionally script its steps in the simulator to generate golden runs and a degraded corpus, and evaluate with `tools/eval_report.py`.

No engine, perception or interface code changes.

10. How it is evaluated

Corpus. Runs are generated by the world-model simulator and then degraded with four profiles — clean, mild, moderate, harsh — that scale and jitter detector confidence, add position noise (up to 1.5 cm per axis), drop frames and occlude the workspace. Each deviation run carries the error injected into it (skip, wrong object, hazard, rush, stall), and each run carries its expected outcome. CSP-1: 130 runs, 346.6 minutes; CRX-2: 70 runs, 58.7 minutes.

Honest scoring. An alarm counts as false if nothing was injected in the run, or if it concerns a step before the injected one — which the injection cannot have caused. The false-alarm rate is normalised by the time in which an alarm could only be false. Alarms about steps after an injected error that genuinely cannot be verified are counted separately as cascades.

Operating point. A sweep over completion and deviation thresholds (`runs/roc_csp1.txt`, `runs/roc_crx2.txt`) confirms 0.85 / 0.85 with 1.5 s persistence: lower completion bars raise CSP-1 false alarms to 8–11 per 45 minutes and produce alarms on permitted reorders.

11. Results

11.1 Headline

See the table in section 1. By deviation type, CSP-1 catches 100 % of skips, hazards and wrong objects and 90 % of timing deviations; CRX-2 catches 100 % of hazards and wrong objects and 95 % of skips. Both experiments raise **zero** false alarms on correct runs, permitted reorders and runs with the camera blocked or the rack markers lost. Steps that could not be verified: 3.9 % (CSP-1) and 7.3 % (CRX-2).

11.2 How the false-alarm target was reached

CSP-1 started at 2.98 false alarms per 45 minutes under honest scoring. Each cause was found by replaying the failing run frame by frame, fixed test-first, and re-measured on the whole corpus; the pinned tests are in `tests/test_false_alarms.py`.

Cause	Mechanism	Fix	Removed
A. Latched evidence thrown away	A momentary action seen before the engine reached its step (the engine was behind the crew) was wiped when the previous step was entered	Trim occurred evidence to the window in which the step could have begun, instead of clearing it	4 false skips (S06)
B. One noisy frame moved a step's start	A single contrary frame counted as "seen still undone", making the next step look rushed	"Seen undone" needs contrary readings unbroken for one independent observation	1 false "too fast"
C. Jitter read as movement	Under heavy noise two halves of a still object's window differ by more than the 3 cm tolerance by chance	Movement is proven only when drift exceeds the tolerance by two	5 false alarms (1

Cause	Mechanism	Fix	Removed
		standard errors of the window's own jitter; otherwise "cannot tell"	skip, 4 timing)

2.98 → 1.92 → **0.85** per 45 minutes (14 → 9 → 4 false alarms), with step accuracy and deviation recall unchanged at every stage.

One further idea was tried and **rejected**: a uniform timing margin cleared a "4.8 s against a 5 s minimum" alarm but also silenced a genuinely rushed step on a clean run, because both measure equally close to their minimum. The guard test caught it and it was reverted.

11.3 What still goes wrong

The failure-mode reel (`runs/reel/reel.mp4`, segments in `docs/reel/segments.json`) shows each with its cause:

- **4 false skips on CSP-1 S07**, all under moderate or harsh degradation. The vial is genuinely moving while it is carried onto the cartridge, which counts against the step; the crew then closes the latch 4.8 s later, 0.2 s inside S07's 5 s minimum. Moving that line would be the same blanket margin that failed above.
- **2 missed rushed steps (S12)**, both under harsh degradation. A blind spot just before the rush leaves the generous reading of its duration at exactly the minimum, so it is not called too fast.
- **Correct steps that read "cannot verify"** on CRX-2 when an occlusion hides the vial right after it is put down — no false alarm, but no confirmation.

12. Performance and endurance

On a laptop CPU (Intel Core 11th gen, 8 threads) at 1280×720, the full live loop — tag detection, rack pose, marker objects, the engine, the display and the overlay — takes 34.6 ms per frame at the median and 74.8 ms at the 95th percentile: **28.9 FPS typical, 13.4 FPS in the slowest 5 % of frames**, against a target of 10 (`runs/bench.json`). The engine alone takes about 1.2 ms per frame.

A 45-minute simulated session (`tools/soak.py --stream`) keeps every temporal history bounded, grows memory by 7.4 MB per hour while recording its trace to disk (`TraceWriter`, byte-identical to the buffered writer), and slows by 1.35× from the first to the last third (limit 1.5×) (`runs/soak_stream.json`). Latency on the test laptop varied up to 3× between runs on identical code because of CPU frequency on battery; comparisons are made within one run or one process.

Power has **not** been measured: the ≤ 25 W target needs a Jetson and a meter. `tools/power_report.py` and `deploy/jetson/` are ready for that measurement.

13. Testing and quality gates

`python tools/check.py` runs every gate and must print **ALL GATES GREEN**: validation of both procedures against the perception build, the zero-retraining check, the validator's negative suite, the JSON schema, the full test suite (602 tests), replay of both golden corpora against their expectations, the evaluation report on the golden runs, and the operator display and voice.

Selected test files:

File	Holds
<code>test_offline.py</code>	No network capability anywhere in the on-board package
<code>test_false_alarms.py</code>	The three fixed false-alarm causes, the guards that keep real detections, and the known limitations as strict expected failures
<code>test_session_window.py</code>	A replay and a live run look identical in the operator window
<code>test_guided.py</code> , <code>test_web.py</code>	The guided runs and the browser app, end to end
<code>test_trace_writer.py</code> , <code>test_clipbuffer.py</code>	Streaming traces are byte-identical; alert clips cover their window or say why not
<code>test_second_procedure.py</code>	CRX-2 runs with no procedure-specific code anywhere

14. Deliverables and how to regenerate them

Deliverable	File	Regenerate
Browser app	<code>demo/</code>	<code>python -m demo</code>
Shareable demo page	<code>runs/demo/parikshak_demo.html</code>	<code>python tools/make_demo_page.py</code>
SIH idea-submission deck	<code>docs/PARIKSHAK_SIH_Idea_Submission.pptx</code> and <code>.pdf</code>	<code>python tools/make_deck.py --render runs/deck/slides</code>
Evaluation	<code>runs/eval.json</code> , <code>runs/eval_crx2.json</code>	<code>python tools/eval_report.py --json runs/eval.json (and --traces traces/eval_crx2 --procedure procedures/crx2_colloid_resuspension.yaml --json runs/eval_crx2.json)</code>

Deliverable	File	Regenerate
Threshold sweep	<code>runs/roc_csp1.txt</code> , <code>runs/roc_crx2.txt</code>	<code>python tools/eval_report.py --sweep</code>
Failure-mode reel	<code>runs/reel/reel.mp4</code>	<code>python tools/make_reel.py</code> <code>docs/reel/segments.json</code>
Downlink sample	<code>runs/downlink_sample/</code>	<code>python tools/make_downlink_sample.py</code>
Benchmark, endurance	<code>runs/bench.json</code> , <code>runs/soak_stream.json</code>	<code>python tools/bench.py</code> , <code>python tools/soak.py --minutes 45 --stream --out runs/soak_stream.json</code>
Printable markers	<code>runs/tags/</code>	<code>python tools/make_tag_sheets.py</code>
4-minute demo script	<code>docs/DEMO_SCRIPT.md</code>	—
Licence statement	<code>docs/LICENCE_STATEMENT.md</code>	—
Deck content, slide by slide	<code>docs/PITCH_DECK.md</code>	—
Build plan	<code>PLAN.md</code>	—

15. Honest limits

- **Synthetic data.** The evaluation runs are generated and degraded, not recorded in orbit or in a laboratory. Real footage will be harder.
- **No trained models yet.** Objects are seen through printed AprilTag markers. Without hand tracking, a live webcam run cannot verify steps that need hands and waits at the first one — without raising a false alarm.
- **Marker precision is measured on rendered frames** (60 mm tags: lateral error 9 mm, depth 34 mm at the 95th percentile); a real webcam will be worse.

- **Edge hardware.** Speed is measured on a laptop CPU only; the Jetson run and the power measurement are not done.
- **Streaming video** to a ground laptop needs GStreamer (planned on the Jetson); on a laptop the flight record and clips are written to disk instead.
- **Voice** uses the system speech engine (pyttsx3); crew commands are typed or clicked, as no speech recogniser is installed.

16. Road to a flight system

1. **Recording campaign** on a mock rack: about 30 correct and 30 error runs, several performers and camera poses, with step boundaries captured at record time.
2. **Models:** an object detector (RT-DETR), hand tracking (MediaPipe or RTMPose), a contact classifier and a motion model, trained on the campaign, MicroG-4M and synthetic renders; evaluated leave-one-performer-out and leave-one-camera-pose-out, plus zero-shot on Assembly101.
3. **Edge:** ONNX → TensorRT INT8 on a Jetson Orin, measured against ≥ 10 FPS at ≤ 25 W on a power meter (`deploy/jetson/README.md`).
4. **Integration:** RTSP to a ground station, Piper voice, spoken commands, multi-camera fusion, and a physically rotated-rack test.
5. **Operations:** cold-boot tests, a spare unit, and rehearsal of the demo script.

The engine, the procedure files and the evaluation harness carry over unchanged; only perception is replaced.

17. Licences

Every runtime library is used under a permissive licence (Apache-2.0, MIT, BSD, MPL-2.0) except PySide6 (LGPL-3.0, dynamically linked, desktop window only). No dataset and no trained weights ship. Delivered weights will be trained only on our own recordings, MicroG-4M (MIT), IndustReal (Apache-2.0) and synthetic renders; CC BY-NC datasets are for benchmark reporting only. Details: `docs/LICENCE_STATEMENT.md`.

18. Glossary

Term	Meaning
Belief frame	One frame of what perception believes: objects, states, hands, confidence, occlusion
Trace	A file of belief frames — a recorded run that can be replayed
PDL	The procedure description language: the experiment as a YAML file

Term	Meaning
Predicate	One condition in a step's verification, such as <code>state_is(latch, closed)</code>
Three-valued truth	A condition's value as an interval, so "unknown" is distinct from "false"
Evidence accumulation	Support and contrary evidence built up over time per step (CUSUM-style)
Duration prior	A step's minimum, nominal and maximum time
Invariant	A safety condition that must hold throughout a step
Frontier	The steps the crew may legally be working on now
Cascade	A correct "cannot verify" on steps after an injected error, not charged as a false alarm
Operating point	The completion and deviation thresholds and persistence the engine runs at
Rack frame	The coordinate frame fixed to the rack by its fiducial markers
Hash chain	Each log record carries the hash of the one before, so any edit is detectable
CFDP	CCSDS File Delivery Protocol, the downlink transport the package is shaped for